

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



SISTEMAS DE COMPUTACIÓN

Trabajo práctico #2:
“Stackframe”

Comisión: Única

Docente: Ing. Javier Alejandro Jorge , Ing. Miguel Ángel Solinas

Alumno: Castilla Juan Felipe - Dalmazzo Agustin

Año lectivo: 2026

Cuatrimestre: Segundo

BASES.....	4
Programa Python.....	4
Interfaz de Capas.....	5
Consumo de la API REST (requests).....	5
Extracción y Filtrado de Datos (Formato json).....	5
Delegación del Procesamiento y Presentación.....	5
Programa en C.....	6
Declaración de Enlace (extern).....	6
Recepción y Truncamiento de Datos.....	6
Saturación de Registros y Desbordamiento al Stack.....	6
Post-procesamiento y Retorno a la Capa Superior.....	7
Programa ASM.....	7
Exportación y Directivas Iniciales (global y section .text).....	7
El Prólogo y Armado del Stack Frame (push rbp / mov rbp, rsp).....	7
Extracción de la Memoria.....	8
mov eax, dword [rbp + 16].....	8
Procesamiento Matemático y Repliegue (add, pop, ret).....	8
DEPURACIÓN.....	8
Verificación de registros.....	10
Inspección de la Dirección de Retorno.....	10
Índice Gini Truncado.....	11

BASES

Para llevar a cabo la actividad propuesta, se trabajará en 3 capas de software, estas son:

- Conexión y solicitud a la API Rest mediante Python
- Comunicación con Asmblle mediante C
- Operaciones lógicas en Asmblle

Mediante estas, y su ejecución en el depurador “GDB” se observará el movimiento de los datos solicitados de la API entre registros de memoria y stack para su posterior truncamiento.

Programa Python

```
➤ MainPython.py > ...
1  import requests
2  import json
3  import ctypes
4
5  # 1. Cargamos la librería compilada de C
6  libcalculos = ctypes.CDLL('./libcalculos.so')
7
8  # 2. Definimos los ti
9  # pos de datos (Python manda un float y C devuelve un entero)
10 libcalculos.procesar_gini.argtypes = (ctypes.c_float,)
11 libcalculos.procesar_gini.restype = ctypes.c_int
12
13 # 3. Definimos el objetivo
14 url_objetivo = "https://api.worldbank.org/v2/en/country/AR/indicator/SI.POV.GINI?format=json&date=2011:2020&per_page=32500&page=1"
15
16 def conectar_api():
17     print("Iniciando conexión con la base de datos objetivo...")
18
19     try:
20         respuesta = requests.get(url_objetivo)
21
22         if respuesta.status_code == 200:
23             print("Conexión establecida con éxito (Estado: 200 OK)\n")
24
25             datos = respuesta.json()
26
27             # La información del Banco Mundial viene en una lista.
28             # En la posición 0 hay metadatos, y en la posición 1 están los registros reales.
29             registros = datos[1]
30
31             print("--- Índice GINI de Argentina (2011-2020) ---")
32
33             # AQUÍ ESTÁ EL FOR: Recorremos cada elemento de la lista de registros
34             for registro in registros:
35                 anio = registro['date']
36                 valor = registro['value']
37
38                 # Algunos años pueden no tener datos (viene como None), los filtramos
39                 if valor is not None:
40                     # Llamamos a nuestro programa en C pasándole el valor flotante
41                     resultado_c = libcalculos.procesar_gini(valor)
42
43                     print(f"Año: {anio} | GINI Original: {valor} | GINI Procesado en Asm (+1): {resultado_c}")
44
45             return datos
46         else:
47             print(f"Fallo en la misión. Código de error: {respuesta.status_code}")
48             return None
49
50     except requests.exceptions.RequestException as e:
51         print(f"Error crítico de red: {e}")
52         return None
53
54 # Ejecución
55 if __name__ == "__main__":
56     datos_crudos = conectar_api()
```

Este script actúa como la capa superior de la arquitectura del sistema. Es el encargado de la orquestación general: obtiene los datos de internet, filtra la información útil y delega el procesamiento matemático a las capas inferiores (C y Ensamblador).

Podemos desglosar su funcionamiento en estos cuatro bloques:

Interfaz de Capas

El programa utiliza ctypes para importar el archivo binario compilado libcalculos.so. Para garantizar la estabilidad de la memoria en la convención de llamadas, le indica al intérprete de Python que debe enviar el dato hacia C como un número de punto flotante (c_float) y que recibirá a cambio un número entero (c_int).

Consumo de la API REST (requests)

Se encarga de establecer la conexión HTTP GET con los servidores del Banco Mundial para descargar la serie histórica del Índice GINI de Argentina (2011-2020). Implementa manejo de errores validando que el código de estado de la respuesta sea 200 (OK) antes de proceder, y captura excepciones de red para evitar caídas abruptas del sistema.

Extracción y Filtrado de Datos (Formato json)

Al recibir la respuesta de la API, el script la decodifica. La estructura de datos del Banco Mundial aloja metadatos en el índice [0] y los datos reales en el índice [1]. El programa aísla la lista de registros y utiliza un bucle for para recorrer la década. Durante la iteración, aplica un filtro condicional (if valor is not None) para descartar aquellos años en los que el Banco Mundial no tiene registros, evitando inyectar datos nulos a la capa de C.

Delegación del Procesamiento y Presentación

Por cada año con datos válidos, Python invoca a la función en C “*procesar_gini(valor)*”, enviando el número flotante. A partir de ese momento, Python pausa su ejecución, espera a que el código en C y Ensamblador hagan el truncamiento y la suma del +1, y captura el entero resultante. Finalmente, imprime en la terminal un resumen formateado mostrando el año, el índice original y el valor procesado a bajo nivel.

Programa en C

```
C SecondLayerC.c > procesar_gini(float)
1 // SecondLayerC.c
2
3 // Declaramos la función en ensamblador
4 extern int procesar_gini_asm(int p1, int p2, int p3, int p4, int p5, int p6, int indice_entero);
5
6 // Esta es la función que Python llama vía ctypes
7 int procesar_gini(float indice_gini) {
8     // Trunca los decimales
9     int indice_entero = (int)indice_gini;
10
11     // Pasa 6 valores basura (10 al 60) a los registros.
12     int resultado = procesar_gini_asm(10, 20, 30, 40, 50, 60, indice_entero);
13     resultado = resultado +9;
14     return resultado;
15 }
```

Este módulo en C actúa como capa intermedia entre la API de alto nivel en Python y la manipulación de hardware en Ensamblador. Su objetivo principal es adaptar los tipos de datos y forzar la convención de llamadas de 64 bits para utilizar la memoria RAM.

Se encuentran cuatro bloques operativos:

Declaración de Enlace (extern)

El código inicia definiendo la firma de la función de ensamblador “*procesar_gini_asm*”. Al declararla como *extern*, le indica al compilador de C que las instrucciones de esta rutina no están en este archivo, sino que serán enlazadas externamente durante la compilación (desde el archivo .o de NASM). Establece el contrato de que la rutina recibirá 7 enteros y devolverá 1 entero.

Recepción y Truncamiento de Datos

La función “*procesar_gini*” es el punto de contacto para Python. Recibe el Índice GINI crudo como un número de punto flotante (float). Inmediatamente ejecuta un *casting* forzado ((*int*)*indice_gini*), lo que le ordena al procesador recortar todos los decimales y almacenar únicamente la parte entera en la nueva variable *indice_entero*.

Saturación de Registros y Desbordamiento al Stack

Según la convención *System V AMD64 ABI*, los primeros 6 parámetros enteros viajan por los registros de propósito general (RDI, RSI, RDX, RCX, R8, R9). Al enviar los números basura del 10 al 60, el programa satura intencionalmente estos registros. Al no quedar espacio, el sistema operativo se ve obligado a empujar el séptimo parámetro (el *indice_entero* real) físicamente hacia la memoria de la pila (Stack) para transferirlo.

Post-procesamiento y Retorno a la Capa Superior

Una vez que el código en Ensamblador ejecuta su ciclo y devuelve el control a C (dejando su cálculo en el registro EAX), C captura ese valor en la variable resultado. Luego, el código aplica una modificación aritmética directa (resultado = resultado + 9;) y ejecuta el comando return para empaquetar el dato final y enviarlo de regreso al script de Python, completando el ciclo de ida y vuelta.

Programa ASM

```
gini.asm
1  global procesar_gini_asm
2
3  section .text
4  procesar_gini_asm:
5      push rbp
6      mov rbp, rsp
7
8      ; En 64-bit Linux:
9      ; RDI, RSI, RDX, RCX, R8, R9 tienen los valores 10, 20, 30, 40, 50, 60
10     ; El 7mo parámetro (el índice GINI real) está en la pila en [rbp + 16]
11
12     mov eax, dword [rbp + 16] ; Trae el índice GINI desde el Stack a EAX
13     add eax, 1                ; Suma 1
14
15     pop rbp
16     ret                      ; Retorna a C (el resultado ya está en EAX)
17
```

Este módulo en Ensamblador puro (NASM) es el núcleo del sistema, la capa más baja donde se opera directamente sobre el hardware del microprocesador y la memoria física.

Se desglosa el código en estos cuatro bloques operativos:

Exportación y Directivas Iniciales (global y section .text)

- *global procesar_gini_asm*: Expone la etiqueta de la función al exterior para que el *linker* (enlazador) la registre y la capa de C pueda localizarla en la memoria al hacer el call.
- *section .text*: Delimita el sector del binario. Le asegura al sistema operativo que las instrucciones siguientes son código de máquina ejecutable y de solo lectura, aislando la lógica de cualquier área de datos.

El Prólogo y Armado del Stack Frame (push rbp / mov rbp, rsp)

Es el protocolo de seguridad necesario.

- *push rbp*: Salva el registro base de la función de C que acaba de llamar a Ensamblador, empujándolo a la pila para no perder su rastro.
- *mov rbp, rsp*: Fija una nueva "ancla" en la memoria copiando el puntero actual de la pila en el registro base. Esto crea un nuevo *Stack Frame* (marco de pila) independiente y seguro para operar sin corromper el entorno del sistema.

Extracción de la Memoria

Sabiendo que los primeros 6 parámetros (del 10 al 60) agotaron la capacidad de los registros de la arquitectura de 64 bits (RDI, RSI, etc.), el código ignora esos registros y va a buscar el 7mo parámetro (el índice GINI) directamente a la RAM.

```
mov eax, dword [rbp + 16]
```

Apuntando exactamente 16 bytes por encima de la base actual, extrae un bloque de 32 bits (dword), y lo carga directamente en el acumulador principal de la CPU (EAX) .

Procesamiento Matemático y Repliegue (add, pop, ret)

- *add eax, 1*: La Unidad Aritmético Lógica (ALU) ejecuta la operación exigida, sumándole +1 al índice GINI. Como el estándar dicta que todo resultado final debe entregarse en el registro EAX, el dato ya queda posicionado para la extracción.
- *pop rbp*: Esta instrucción extrae el valor original del puntero base que se guardó en el prólogo y lo reinserta en el registro *RBP*, restaurando la estructura espacial de la memoria de C exactamente como estaba antes de la invocación..
- *ret*: Ejecuta el salto de regreso. La CPU lee la dirección de retorno que estaba guardada en el Stack y le devuelve el mando a la capa intermedia con el resultado matemático cargado y listo en el registro.

DEPURACIÓN

Iniciamos la depuración del programa, empezaremos colocando el freno directamente en la función de Ensamblador. Como la librería de C se carga dinámicamente, GDB solicitará confirmación para dejar el *breakpoint* en espera.

- `gdb python3`

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion/TP2/SdeC-IE-2026-G-La-popu-92.3-TP2$ gdb python3
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
```

- break SecondLayerC.c:10
- break SecondLayerC.c:12
- break procesar_gini_asm

```
>>> break SecondLayerC.c:10
No hay tabla de símbolos cargada. Use la orden «file».
Make breakpoint pending on future shared library load? (y or [n]) y
Punto de interrupción 1 (SecondLayerC.c:10) pendiente.
>>> break SecondLayerC.c:12
No hay tabla de símbolos cargada. Use la orden «file».
Make breakpoint pending on future shared library load? (y or [n]) y
Punto de interrupción 2 (SecondLayerC.c:12) pendiente.
>>> break procesar_gini_asm
Función «procesar_gini_asm» no definida.
Make breakpoint pending on future shared library load? (y or [n]) y
Punto de interrupción 3 (procesar_gini_asm) pendiente.
```

- run MainPython.py

```
Breakpoint 1, procesar_gini (indice_gini=42.7000008) at SecondLayerC.c:12
12 int resultado = procesar_gini_asm(10, 20, 30, 40, 50, 60, indice_entero);

Assembly
0x00007ffff7a5a111 procesar_gini+8 sub $0x20,%rsp
0x00007ffff7a5a115 procesar_gini+12 movss %xmm0,-0x14(%rbp)
0x00007ffff7a5a11a procesar_gini+17 movss -0x14(%rbp),%xmm0
0x00007ffff7a5a11f procesar_gini+22 cvttss2si %xmm0,%eax
0x00007ffff7a5a123 procesar_gini+26 mov %eax,-0x8(%rbp)
!0x00007ffff7a5a126 procesar_gini+29 sub $0x8,%rsp
0x00007ffff7a5a12a procesar_gini+33 mov -0x8(%rbp),%eax
0x00007ffff7a5a12d procesar_gini+36 push %rax
0x00007ffff7a5a12e procesar_gini+37 mov $0x3c,%r9d
0x00007ffff7a5a134 procesar_gini+43 mov $0x32,%r8d

Breakpoints
[1] break at 0x00007ffff7a5a126 in SecondLayerC.c:12 for SecondLayerC.c:12 hit 1 time
[2] break at 0x00007ffff7a5a174 for procesar_gini_asm
[3] break at 0x00007ffff7a5a126 in SecondLayerC.c:12 for SecondLayerC.c:10 hit 1 time

Expressions
History
Memory
Registers
rax 0x000000000000002a rbx 0x0000000000000001 rcx 0x0000000000000001 rdx 0x0000000000551110
rsi 0x0000000000584b58 rdi 0x00007fffffd450 rbp 0x00007fffffd4b0 rsp 0x00007fffffd490
r8 0x00007fffffd430 r9 0x00007ffff6370810 r10 0x0000000000000000 r11 0x00007ffff7a5a109
r12 0x00007fffffd668 r13 0x00007fffffd730 r14 0x00007fffffd580 r15 0x00007fffffd658
rip 0x00007ffff7a5a126 eflags [ PF IF ] cs 0x00000033 ss 0x0000002b
ds 0x00000000 es 0x00000000 fs 0x00000000 gs 0x00000000
fs_base 0x00007ffff7e6a080 gs_base 0x0000000000000000

Source
7 int procesar_gini(float indice_gini) {
8 // Trunca los decimales
9 int indice_entero = (int)indice_gini;
10
11 // Pasa 6 valores basura (10 al 60) a los registros.
!12 int resultado = procesar_gini_asm(10, 20, 30, 40, 50, 60, indice_entero);
13 resultado = resultado +9;
14 return resultado;
15 }

Stack
[0] from 0x00007ffff7a5a126 in procesar_gini+29 at SecondLayerC.c:12
[1] from 0x00007ffff7ab7b16
[2] from 0x00007ffff7ab43ef
[3] from 0x00007ffff7ab70be in ffi_call
[4] from 0x00007ffff7ac11c
[5] from 0x00007ffff7ac52af
[6] from 0x0000000000548f35 in PyObject_MakeTpCall
[7] from 0x00000000005d6f09 in PyEval_EvalFrameDefault
[8] from 0x00000000005d544b in PyEval_EvalCode
[9] from 0x00000000006085a2
[+]

Threads
[1] id 7698 name python3 from 0x00007ffff7a5a126 in procesar_gini+29 at SecondLayerC.c:12

Variables
arg indice_gini = 42.7000008
loc indice_entero = 42, resultado = 0
```

El programa hará la petición web, entrará a C y se congelará justo al inicio de Ensamblador.

Verificación de registros

Solicitaremos los registros antes de que sean modificados por la petición de la segunda capa en C

- `info registers rdi rsi rdx rcx r8 r9` (se verán los valores en formato hexadecimal: `0xa`, `0x14`, `0x1e`, `0x28`, `0x32`, `0x3c`).

```
>>> info registers rdi rsi rdx rcx r8 r9
rdi      0x7fffffff450      140737488344144
rsi      0x584b58          5786456
rdx      0x551110          5574928
rcx      0x1               1
r8       0x7fffffff430      140737488344112
r9       0x7ffff6370810     140737324189712
>>> █
```

Luego, mediante “stepi” avanzaremos el programa viendo como se llenan uno a uno los registros con los valores 10,20,30,40,50,60 y por último “Índice” irá al Stack

Los encargados de esto son las líneas que vemos por encima

```
0x00007ffff7a5a12e procesar_gini+37 mov $0x3c,%r9d
0x00007ffff7a5a134 procesar_gini+43 mov $0x32,%r8d
0x00007ffff7a5a13a procesar_gini+49 mov $0x28,%ecx
0x00007ffff7a5a13f procesar_gini+54 mov $0x1e,%edx
0x00007ffff7a5a144 procesar_gini+59 mov $0x14,%esi
```

donde se encuentran los valores decimales en formato hexadecimal, si usamos stepi, podremos ver como se cargan los registros paso a paso.

```
>>> info registers rdi rsi rdx rcx r8 r9
rdi      0xa               10
rsi      0x14              20
rdx      0x1e              30
rcx      0x28              40
r8       0x32              50
r9       0x3c              60
```

vamos a continuar el programa.

Inspección de la Dirección de Retorno

Para saber a dónde volverá el programa una vez que Ensamblador termine, debes mirar la memoria de la pila. El comando `call` guardó esta dirección justo antes de entrar a tu rutina.

```
0x00007ffff7a5a14e procesar_gini+69 call 0x7ffff7a5a030 <procesar_gini_asm@plt>
0x00007ffff7a5a153 procesar_gini+74 add $0x10,%rsp
```

este comando es el siguiente a la carga de los registros

Al ejecutarlo , la CPU empuja automáticamente la dirección de retorno a la pila, es decir, este comando nos lleva al programa de Assembler donde sumamos uno al índice GINI.

Ejecutamos “c” en gdb para que el programa continúe hacia el próximo breakpoint (*procesar_gini_asm*), aquí veremos la dirección de retorno mediante *x/1xg \$rbp+8* , dado que nos devuelve una dirección en hexadecimal, ejecutamos *x/6i [Dirección en Hexadecimal]* para que nos muestre las siguientes 6 instrucciones que se ejecutarán luego de volver a esta, se observará lo siguiente:

```
>>> x/1xg $rbp+8
0x7fffffd478: 0x00007ffff7a5a153
>>> x/6i 0x00007ffff7a5a153
0x7ffff7a5a153 <procesar_gini+74>: add $0x10,%rsp
0x7ffff7a5a157 <procesar_gini+78>: mov %eax,-0x4(%rbp)
0x7ffff7a5a15a <procesar_gini+81>: addl $0x9,-0x4(%rbp)
0x7ffff7a5a15e <procesar_gini+85>: mov -0x4(%rbp),%eax
0x7ffff7a5a161 <procesar_gini+88>: leave
0x7ffff7a5a162 <procesar_gini+89>: ret
>>> █
```

Nuestra dirección de retorno será: *0x00007ffff7a5a153*

Aquí se encuentran las acciones que realiza nuestro programa en C, se puede ver que la línea

```
0x7ffff7a5a15a <procesar_gini+81>: addl $0x9,-0x4(%rbp)
```

Sería nuestro “*Resultado = Resultado+9*” traducido al lenguaje Assembler. He aquí nuestro análogo en C.

```
// Pasa 6 valores basura (10 al 60) a los registros.
int resultado = procesar_gini_asm(10, 20, 30, 40, 50, 60, indice_entero);
resultado = resultado +9;
return resultado;
```

Índice Gini Truncado

Por último, veremos cómo se encuentra el índice gini en este punto, pues ya debería de estar truncado. Se ejecuta

```
x/1xg $rbp+16
```

Esto nos devolverá el valor en Hexadecimal, si en cambio usamos

`x/1dw $rbp+16`

Podremos ver su valor en formato Decimal.

```
>>> x/1xg $rbp+16
0x7fffffffdd480: 0x0000000000000002a
>>> x/1dw $rbp+16
0x7fffffffdd480: 42
>>>
```

Recordemos que en el programa assemble se le suma uno y en el programa en C se adicionan 9 enteros más, esta suma en C se usa con el objetivo de revelar la posición de retorno al programa.

```
Año: 2011 | GINI Original: 42.7 | GINI Procesado en Asm (+1): 52
```